

Lecture 7: Normal Distribution, Joint PDFs

Math 151: Probability

Jamie Haddock

Table of contents

1	Normal Distribution	1
1.1	Why the normal distribution?	1
1.2	Normal random variables: definition	2
1.2.1	Mean, variance, and a bit of intuition	2
1.2.2	Normality is preserved by linear transformations	3
1.3	The standard normal random variable	4
1.3.1	Standardizing a normal random variable	5
1.3.2	Example: snowfall	5
1.3.3	Example: signal detection	6
2	Joint PDFs	7
2.1	Joint PDF of continuous random variables	7
2.1.1	Example: a 2D uniform density	8
2.1.2	Example: Buffon's needle	9
2.2	Joint CDFs	10
2.2.1	Example: uniform on the unit square	10
2.3	Expectation for jointly continuous RVs	11
2.4	Extending to more than two random variables	12

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from scipy.integrate import quad, dblquad
rng = np.random.default_rng(151)
plt.rcParams["figure.dpi"] = 110
```

1 Normal Distribution

1.1 Why the normal distribution?

Last time we met two continuous distributions built from very concrete stories: the **Uniform** (equally likely along an interval) and the **Exponential** (waiting times, memorylessness).

The **Normal** (or **Gaussian**) distribution is different: it shows up not because of one specific story, but because it is the *universal limit* of averaging many small, independent effects — measurement error, heights, exam scores, noise in a communication channel. We'll make this precise later in the course (the **Central Limit Theorem**), but for now we introduce it as a named continuous distribution, exactly like we did for Uniform and Exponential.

It is, by a wide margin, the most important distribution in probability and statistics.

1.2 Normal random variables: definition

Definition: Normal random variable

A continuous random variable X is **normal** (or **Gaussian**) with parameters $\mu \in \mathbb{R}$ and $\sigma^2 > 0$, written $X \sim N(\mu, \sigma^2)$, if it has PDF

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad x \in \mathbb{R}.$$

- The density is a symmetric “bell curve” centered at μ .
- σ controls the spread: small σ gives a tall, narrow bump; large σ gives a short, wide one.
- It’s not obvious from the formula alone that this integrates to 1 — that relies on the classical **Gaussian integral** $\int_{-\infty}^{\infty} e^{-u^2/2} du = \sqrt{2\pi}$, which we’ll take as known and verify numerically below.

```
# Visualizing the PDF and CDF for a few choices of (mu, sigma)
```

```
xs = np.linspace(-10, 14, 500)
```

```
params = [(0, 1), (2, 1), (2, 3)]
```

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
```

```
for mu, sigma in params:
```

```
    axes[0].plot(xs, stats.norm.pdf(xs, mu, sigma), label=fr"$\mu={mu}$, \sigma={sigma}$")
```

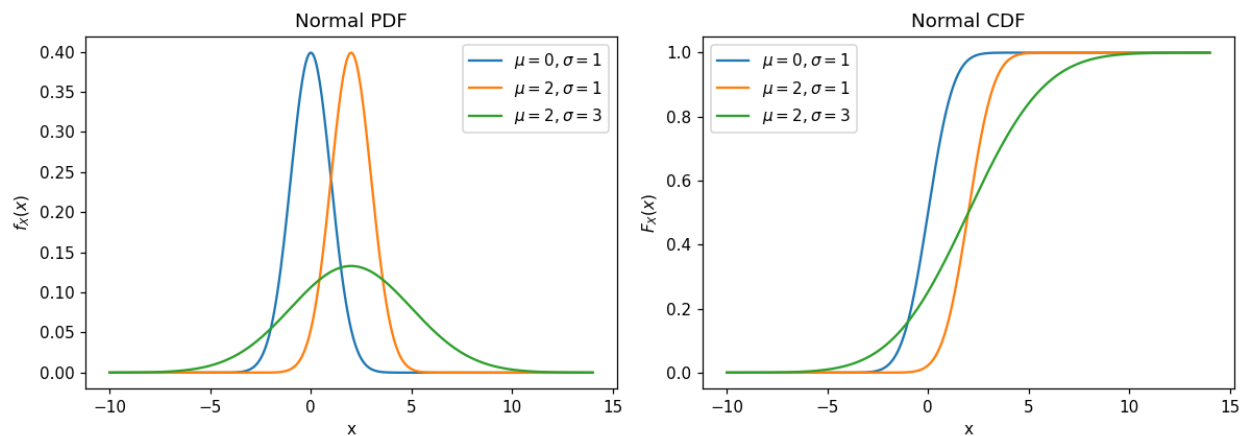
```
    axes[1].plot(xs, stats.norm.cdf(xs, mu, sigma), label=fr"$\mu={mu}$, \sigma={sigma}$")
```

```
axes[0].set_title("Normal PDF"); axes[0].set_xlabel("x"); axes[0].set_ylabel(r"$f_X(x)$"); axes[0].legend()
```

```
axes[1].set_title("Normal CDF"); axes[1].set_xlabel("x"); axes[1].set_ylabel(r"$F_X(x)$"); axes[1].legend()
```

```
plt.tight_layout()
```

```
plt.show()
```



1.2.1 Mean, variance, and a bit of intuition

Fact

If $X \sim N(\mu, \sigma^2)$, then

$$E[X] = \mu, \quad \text{Var}(X) = \sigma^2.$$

Why $E[X] = \mu$: substitute $u = x - \mu$. The density becomes symmetric in u about 0, i.e. $f_X(\mu + u) = f_X(\mu - u)$, and

$$E[X] - \mu = E[X - \mu] = \int_{-\infty}^{\infty} u \cdot \frac{1}{\sigma\sqrt{2\pi}} e^{-u^2/(2\sigma^2)} du = 0,$$

because the integrand is an **odd function** of u (odd $\times$ even = odd), and the integral of an odd function over $(-\infty, \infty)$ vanishes. So $E[X] = \mu$.

Why $\text{Var}(X) = \sigma^2$: this takes an integration-by-parts argument on $\int u^2 e^{-u^2/(2\sigma^2)} du$ that we won't carry out in full generality here — but let's at least *check* both facts numerically for a specific case, alongside the fact that the density integrates to 1 at all.

```
mu, sigma = 2.0, 3.0
f = lambda x: stats.norm.pdf(x, mu, sigma)

total_mass, _ = quad(f, -np.inf, np.inf)
mean, _ = quad(lambda x: x * f(x), -np.inf, np.inf)
var, _ = quad(lambda x: (x - mu) ** 2 * f(x), -np.inf, np.inf)

print(f"integral of f_X          = {total_mass:.6f}    (should be 1)")
print(f"E[X] (via quad)          = {mean:.6f}        (should be mu = {mu})")
print(f"Var(X) (via quad)         = {var:.6f}         (should be sigma^2 = {sigma**2})")

integral of f_X          = 1.000000    (should be 1)
E[X] (via quad)          = 2.000000    (should be mu = 2.0)
Var(X) (via quad)         = 9.000000    (should be sigma^2 = 9.0)
```

1.2.2 Normality is preserved by linear transformations

Theorem

If $X \sim N(\mu, \sigma^2)$ and $Y = aX + b$ for constants $a \neq 0, b$, then

$$Y \sim N(a\mu + b, a^2\sigma^2).$$

Proof

Recall from Lecture 4/6 the PDF-of-a-transform formula: if g is invertible and differentiable with inverse $h = g^{-1}$, then $f_Y(y) = f_X(h(y)) |h'(y)|$. Here $g(x) = ax + b$, so $h(y) = (y - b)/a$ and $h'(y) = 1/a$. Then

$$f_Y(y) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{\left(\frac{y-b}{a} - \mu\right)^2}{2\sigma^2}\right) \cdot \frac{1}{|a|} = \frac{1}{|a|\sigma\sqrt{2\pi}} \exp\left(-\frac{(y - (a\mu + b))^2}{2a^2\sigma^2}\right),$$

which is exactly the $N(a\mu + b, a^2\sigma^2)$ density (note $|a|\sigma = \sqrt{a^2\sigma^2}$). ■

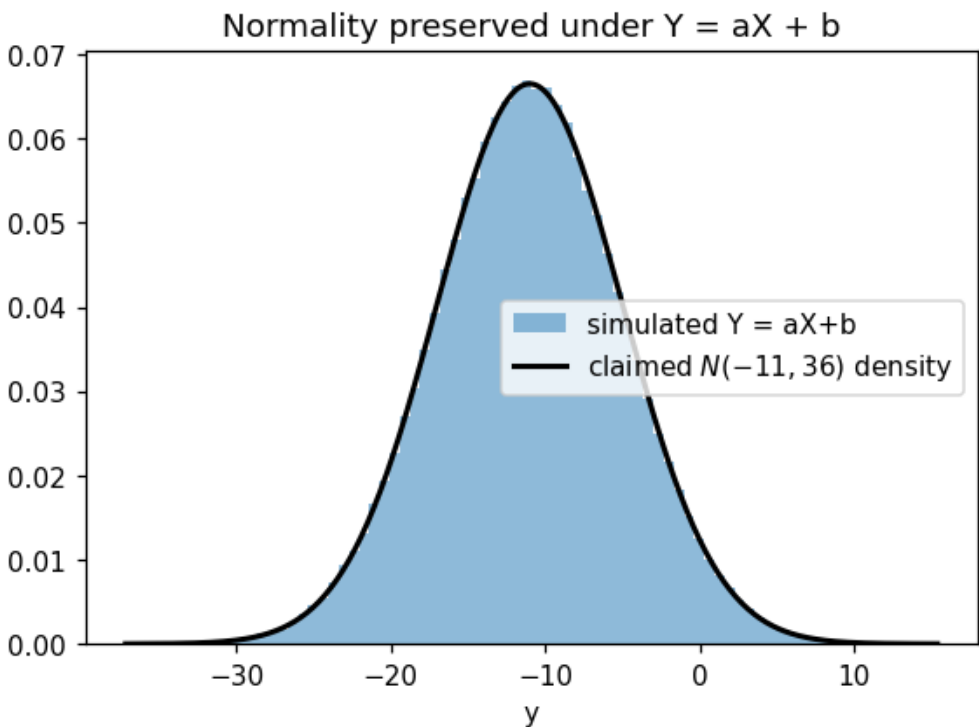
A linear function of a normal random variable is *again normal* — this is a special structural property that most distributions don't have (compare: a linear function of a Uniform or Exponential RV is generally *not* Uniform or Exponential).

```
# Numerical check: simulate Y = aX + b and compare its histogram to the claimed N(a*mu+b, a^2 sigma^2)
mu, sigma, a, b = 5.0, 2.0, -3.0, 4.0
X = rng.normal(mu, sigma, size=200_000)
Y = a * X + b

y_grid = np.linspace(Y.min(), Y.max(), 400)
claimed_mean, claimed_var = a * mu + b, a**2 * sigma**2
```

```
plt.figure(figsize=(6, 4))
plt.hist(Y, bins=80, density=True, alpha=0.5, label="simulated Y = aX+b")
plt.plot(y_grid, stats.norm.pdf(y_grid, claimed_mean, np.sqrt(claimed_var)),
        'k-', lw=2, label=fr"claimed $N(\{claimed\_mean:.0f\}, \{claimed\_var:.0f\})$ density")
plt.legend(); plt.xlabel("y"); plt.title("Normality preserved under Y = aX + b")
plt.show()

print(f"sample mean of Y = {Y.mean():.3f} (claimed {claimed_mean})")
print(f"sample var of Y = {Y.var():.3f} (claimed {claimed_var})")
```



```
sample mean of Y = -11.012 (claimed -11.0)
sample var of Y = 36.040 (claimed 36.0)
```

1.3 The standard normal random variable

i Definition: Standard normal

$Z \sim N(0, 1)$ is the **standard normal** random variable. Its PDF and CDF get their own names:

$$\varphi(z) = \frac{1}{\sqrt{2\pi}} e^{-z^2/2}, \quad \Phi(z) = \int_{-\infty}^z \varphi(t) dt.$$

Φ has no closed form in terms of elementary functions — historically this is why every probability textbook has a **standard normal table** in the back, giving $\Phi(z)$ for a grid of z values. A short excerpt:

z	0.00	0.50	1.00	1.50	1.645	2.00
$\Phi(z)$	0.5000	0.6915	0.8413	0.9332	0.9500	0.9772

Today we'll use `scipy.stats.norm` to play the role of the table, but the *method* — standardize, then look up — is exactly what you'd do with a printed table (and what you should be able to do on an exam with one).

1.3.1 Standardizing a normal random variable

💡 Corollary (of the linear-transformation theorem)

If $X \sim N(\mu, \sigma^2)$, then

$$Z = \frac{X - \mu}{\sigma} \sim N(0, 1).$$

This is immediate from the theorem two slides ago with $a = 1/\sigma$, $b = -\mu/\sigma$: $a\mu + b = 0$ and $a^2\sigma^2 = 1$.

Why this matters: it lets us answer *any* probability question about *any* normal $X \sim N(\mu, \sigma^2)$ using only the single function Φ :

$$P(X \leq x) = P\left(\frac{X - \mu}{\sigma} \leq \frac{x - \mu}{\sigma}\right) = \Phi\left(\frac{x - \mu}{\sigma}\right).$$

1.3.2 Example: snowfall

Annual snowfall X (in inches) at a mountain resort is modeled as $X \sim N(60, 20^2)$.

(a) What's the probability of more than 90 inches of snow this year?

$$P(X > 90) = 1 - \Phi\left(\frac{90 - 60}{20}\right) = 1 - \Phi(1.5).$$

(b) How much snow is exceeded in only 5% of years — i.e. find x^* with $P(X > x^*) = 0.05$?

$$1 - \Phi\left(\frac{x^* - 60}{20}\right) = 0.05 \iff \Phi\left(\frac{x^* - 60}{20}\right) = 0.95 \iff \frac{x^* - 60}{20} = 1.645$$

using the table value $\Phi(1.645) \approx 0.95$ — so $x^* \approx 60 + 20(1.645) = 92.9$ inches.

```
mu, sigma = 60.0, 20.0

# (a)
p_over_90 = 1 - stats.norm.cdf((90 - mu) / sigma)
print(f"(a) P(X > 90) = {p_over_90:.4f}    [table method: 1 - Phi(1.5) = {1 - 0.9332:.4f}]")

# (b) 95th percentile
z95 = stats.norm.ppf(0.95)
x_star = mu + sigma * z95
print(f"(b) z with Phi(z)=0.95: {z95:.3f}")
print(f"    x* (95th percentile) = {x_star:.2f} inches")
```

(a) $P(X > 90) = 0.0668$ [table method: $1 - \Phi(1.5) = 0.0668$]

(b) z with $\Phi(z)=0.95$: 1.645
 x^* (95th percentile) = 92.90 inches

1.3.3 Example: signal detection

A transmitter sends a bit $s \in \{-1, +1\}$ (equally likely) over a noisy channel. The receiver observes

$$Y = s + N, \quad N \sim N(0, \sigma^2) \text{ independent of } s,$$

and decides “+1 was sent” if $Y > 0$, and “−1 was sent” otherwise.

Question

What is the probability the receiver makes an error?

By symmetry it suffices to compute the error probability given $s = +1$ was sent (the case $s = -1$ is identical by symmetry of the noise):

$$P(\text{error} \mid s = 1) = P(Y < 0 \mid s = 1) = P(1 + N < 0) = P(N < -1) = \Phi\left(\frac{-1}{\sigma}\right).$$

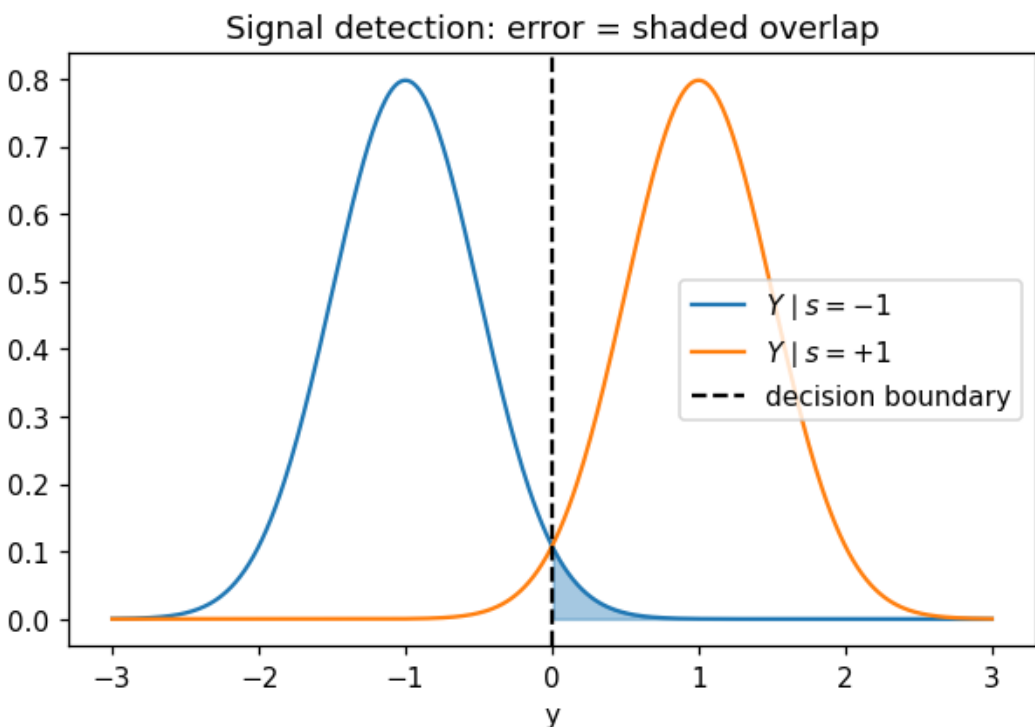
So $P(\text{error}) = \Phi(-1/\sigma)$ — smaller noise σ (relative to the signal strength 1) means a more reliable channel, exactly as intuition demands.

```
sigma = 0.5
p_error_formula = stats.norm.cdf(-1 / sigma)
print(f"P(error) = Phi(-1/sigma) = {p_error_formula:.4f}    (sigma = {sigma})")

# Verify by simulation
n = 2_000_000
s = rng.choice([-1, 1], size=n)
N = rng.normal(0, sigma, size=n)
Y = s + N
decision = np.where(Y > 0, 1, -1)
p_error_sim = np.mean(decision != s)
print(f"P(error), simulated = {p_error_sim:.4f}")

# Visualize the two conditional densities and the decision boundary
y_grid = np.linspace(-3, 3, 400)
plt.figure(figsize=(6.5, 4))
plt.plot(y_grid, stats.norm.pdf(y_grid, -1, sigma), label=r"$Y \mid s=-1$")
plt.plot(y_grid, stats.norm.pdf(y_grid, 1, sigma), label=r"$Y \mid s=+1$")
plt.axvline(0, color="k", linestyle="--", label="decision boundary")
plt.fill_between(y_grid[y_grid > 0], 0, stats.norm.pdf(y_grid[y_grid > 0], -1, sigma),
                 alpha=0.4, color="C0")
plt.legend(); plt.xlabel("y"); plt.title("Signal detection: error = shaded overlap")
plt.show()
```

```
P(error) = Phi(-1/sigma) = 0.0228    (sigma = 0.5)
P(error), simulated = 0.0228
```



2 Joint PDFs

2.1 Joint PDF of continuous random variables

i Definition: Joint PDF

Random variables X, Y are **jointly continuous** with **joint PDF** $f_{X,Y}(x, y)$ if $f_{X,Y}(x, y) \geq 0$ everywhere,

$$\iint_{\mathbb{R}^2} f_{X,Y}(x, y) dx dy = 1,$$

and for every (nice) region $B \subseteq \mathbb{R}^2$,

$$P((X, Y) \in B) = \iint_B f_{X,Y}(x, y) dx dy.$$

This is the direct continuous analogue of the joint PMF from Lecture 5 — sums become double integrals.

i Definition: Marginal PDF

$$f_X(x) = \int_{-\infty}^{\infty} f_{X,Y}(x, y) dy, \quad f_Y(y) = \int_{-\infty}^{\infty} f_{X,Y}(x, y) dx.$$

Exactly like Lecture 5's marginal PMF: “integrate out” the variable you don’t want, the continuous version of “sum out.”

2.1.1 Example: a 2D uniform density

Let (X, Y) be **uniform on the unit disk** $D = \{(x, y) : x^2 + y^2 \leq 1\}$, i.e.

$$f_{X,Y}(x, y) = \begin{cases} \frac{1}{\pi} & (x, y) \in D \\ 0 & \text{otherwise,} \end{cases}$$

since the disk has area π . What is the marginal $f_X(x)$?

$$f_X(x) = \int_{-\sqrt{1-x^2}}^{\sqrt{1-x^2}} \frac{1}{\pi} dy = \frac{2\sqrt{1-x^2}}{\pi}, \quad -1 \leq x \leq 1.$$

Warning

Even though the **joint** density is perfectly flat (uniform), the **marginal** density of X alone is *not* uniform — it's largest at $x = 0$ (where the disk is widest) and tapers to 0 at $x = \pm 1$. Marginalizing a uniform joint density does not generally give a uniform marginal!

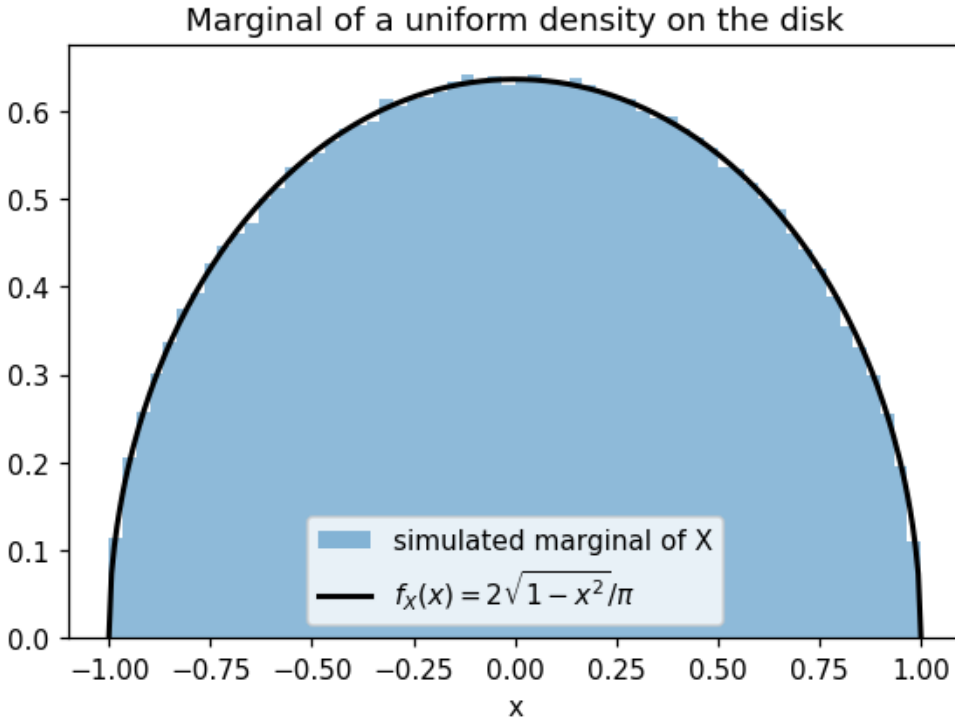
```
# Verify the marginal formula: numerical integration vs. rejection-sampling simulation
f_X_formula = lambda x: 2 * np.sqrt(1 - x**2) / np.pi

# check it integrates to 1
mass, _ = quad(f_X_formula, -1, 1)
print(f"integral of claimed f_X over [-1,1] = {mass:.6f} (should be 1)")

# simulate uniform points in the disk by rejection sampling from the bounding square
n = 500_000
pts = rng.uniform(-1, 1, size=(3 * n, 2))
inside = pts[pts[:, 0]**2 + pts[:, 1]**2 <= 1][:n]
X_samples = inside[:, 0]

x_grid = np.linspace(-1, 1, 300)
plt.figure(figsize=(6, 4))
plt.hist(X_samples, bins=60, density=True, alpha=0.5, label="simulated marginal of X")
plt.plot(x_grid, f_X_formula(x_grid), 'k-', lw=2, label=r"$f_X(x) = 2\sqrt{1-x^2}/\pi$")
plt.legend(); plt.xlabel("x"); plt.title("Marginal of a uniform density on the disk")
plt.show()
```

```
integral of claimed f_X over [-1,1] = 1.000000 (should be 1)
```



2.1.2 Example: Buffon's needle

A floor is ruled with parallel lines a distance d apart. A needle of length $L \leq d$ is dropped at random. What's the probability it crosses a line?

Setup. Let X = distance from the needle's center to the nearest line, and Θ = acute angle the needle makes with the lines. A "random drop" means

$$X \sim \text{Uniform}(0, d/2), \quad \Theta \sim \text{Uniform}(0, \pi/2), \quad X, \Theta \text{ independent},$$

so the joint PDF is $f_{X,\Theta}(x, \theta) = \frac{1}{(d/2)} \cdot \frac{1}{(\pi/2)} = \frac{4}{\pi d}$ on the rectangle $[0, d/2] \times [0, \pi/2]$.

The needle **crosses a line** exactly when $X \leq \frac{L}{2} \sin \Theta$ (the needle's half-length, projected perpendicular to the lines, reaches past the center to the line). So

$$P(\text{cross}) = \iint_{\{x \leq \frac{L}{2} \sin \theta\}} \frac{4}{\pi d} dx d\theta = \frac{4}{\pi d} \int_0^{\pi/2} \frac{L}{2} \sin \theta d\theta = \frac{4}{\pi d} \cdot \frac{L}{2} \cdot 1 = \frac{2L}{\pi d}.$$

💡 Tip

Solving for π : $\pi = \frac{2L}{d \cdot P(\text{cross})}$ — dropping needles gives a (very slow!) randomized way to estimate π .

```
L, d = 1.0, 2.0
p_cross_formula = 2 * L / (np.pi * d)
print(f"P(cross), formula = {p_cross_formula:.5f}")
```

```

n = 2_000_000
X = rng.uniform(0, d / 2, size=n)
Theta = rng.uniform(0, np.pi / 2, size=n)
crosses = X <= (L / 2) * np.sin(Theta)
p_cross_sim = crosses.mean()
pi_estimate = 2 * L / (d * p_cross_sim)

print(f"P(cross), simulated = {p_cross_sim:.5f} (n = {n:,} drops)")
print(f"implied estimate of pi = {pi_estimate:.4f} (true pi = {np.pi:.4f})")

P(cross), formula = 0.31831
P(cross), simulated = 0.31821 (n = 2,000,000 drops)
implied estimate of pi = 3.1426 (true pi = 3.1416)

```

2.2 Joint CDFs

i Definition: Joint CDF

$$F_{X,Y}(x,y) = P(X \leq x, Y \leq y) = \int_{-\infty}^x \int_{-\infty}^y f_{X,Y}(u,v) dv du.$$

Same idea as a single-variable CDF: “accumulated probability,” now over a **quadrant** $(-\infty, x] \times (-\infty, y]$ rather than a ray.

2.2.1 Example: uniform on the unit square

Let (X,Y) be uniform on $[0,1]^2$, so $f_{X,Y}(x,y) = 1$ on the square (note: here X,Y are independent Uniform(0,1)’s, unlike the disk example). For $0 \leq x, y \leq 1$,

$$F_{X,Y}(x,y) = \int_0^x \int_0^y 1 dv du = xy.$$

Visual interpretation: since the density is flat (height 1), $F_{X,Y}(x,y)$ is just the probability mass in the rectangle $[0,x] \times [0,y]$, which — because density is 1 — equals the *area* of that rectangle, $x \cdot y$.

```

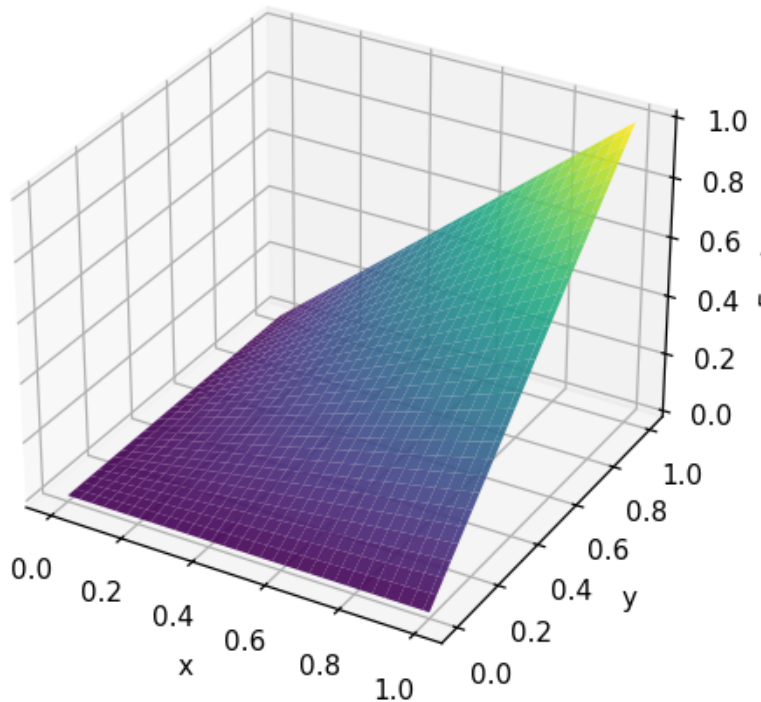
# Visualize F(x,y) = xy over the unit square, and check it against numerical integration of f=1
xg = np.linspace(0, 1, 60)
yg = np.linspace(0, 1, 60)
Xg, Yg = np.meshgrid(xg, yg)
Fg = Xg * Yg

fig = plt.figure(figsize=(6, 5))
ax = fig.add_subplot(projection="3d")
ax.plot_surface(Xg, Yg, Fg, cmap="viridis", alpha=0.9)
ax.set_xlabel("x"); ax.set_ylabel("y"); ax.set_zlabel(r"$F_{\{X,Y\}}(x,y)$")
ax.set_title("Joint CDF of Uniform on the unit square")
plt.show()

# spot check at one point via double integration of the density
x0, y0 = 0.3, 0.7
F_numeric, _ = dblquad(lambda v, u: 1.0, 0, x0, 0, y0)
print(f"F(0.3, 0.7) via dblquad = {F_numeric:.4f} (formula x*y = {x0*y0:.4f})")

```

Joint CDF of Uniform on the unit square



$F(0.3, 0.7)$ via `dblquad` = 0.2100 (formula $x*y$ = 0.2100)

2.3 Expectation for jointly continuous RVs

i Definition: Two-variable LOTUS

$$E[g(X, Y)] = \iint_{\mathbb{R}^2} g(x, y) f_{X,Y}(x, y) dx dy.$$

The exact continuous counterpart of the two-variable LOTUS from Lecture 5 (sums $\rightarrow$ double integrals).

💡 Fact: Linearity of expectation

For any jointly continuous X, Y (independent or not!) and constants a, b ,

$$E[aX + bY] = aE[X] + bE[Y].$$

Just as in the discrete case (Lecture 4–5), this needs **no independence assumption** — it follows immediately by taking $g(x, y) = ax + by$ in the two-variable LOTUS and splitting the double integral.

```
# Check linearity of expectation on the Uniform-on-unit-square example: E[aX+bY] = a*E[X] + b*E[Y]
a, b = 3.0, -2.0
```

```
EX, _ = dblquad(lambda y, x: x * 1.0, 0, 1, 0, 1)
EY, _ = dblquad(lambda y, x: y * 1.0, 0, 1, 0, 1)
E_lin, _ = dblquad(lambda y, x: (a * x + b * y) * 1.0, 0, 1, 0, 1)
```

```

print(f"E[X] = {EX:.4f}, E[Y] = {EY:.4f}")
print(f"E[aX+bY] via LOTUS directly = {E_lin:.4f}")
print(f"a*E[X] + b*E[Y]                = {a*EX + b*EY:.4f}")

# and by simulation
n = 1_000_000
Xs = rng.uniform(0, 1, n)
Ys = rng.uniform(0, 1, n)
print(f"E[aX+bY], simulated            = {np.mean(a*Xs + b*Ys):.4f}")

```

```

E[X] = 0.5000, E[Y] = 0.5000
E[aX+bY] via LOTUS directly = 0.5000
a*E[X] + b*E[Y]              = 0.5000
E[aX+bY], simulated          = 0.4990

```

2.4 Extending to more than two random variables

Everything above extends directly to a collection $X_1, \dots, X_n$: a joint PDF $f_{X_1, \dots, X_n}$ on $\mathbb{R}^n$ that integrates to 1, marginals obtained by integrating out the variables you don't want, a joint CDF, an n -variable LOTUS, and linearity of expectation for *any* linear combination $\sum_i a_i X_i$ — again with no independence required.

i A first look: multivariate Gaussian random variables

A random vector $(X_1, \dots, X_n)$ is (jointly, or **multivariate**) **Gaussian** if every linear combination $\sum_i a_i X_i$ is a (univariate) normal random variable. We write $X \sim N(\mu, \Sigma)$, where $\mu = (E[X_1], \dots, E[X_n])$ and Σ is the **covariance matrix** — a natural generalization of σ^2 that also records how the X_i vary *together*.

Two structural facts worth previewing (we'll make “covariance” precise in a few weeks, when we get to independence and correlation of random variables):

- each individual X_i is itself normal (a marginal of a multivariate Gaussian is Gaussian);
- unlike most joint distributions, the *entire* joint distribution of a multivariate Gaussian is pinned down by just the means and the covariance matrix Σ .

```

# A visual teaser: a 2D Gaussian with correlated components
rho = 0.6
mean = [0, 0]
cov = [[1, rho], [rho, 1]]

x, y = np.mgrid[-3:3:.05, -3:3:.05]
pos = np.dstack((x, y))
rv = stats.multivariate_normal(mean, cov)

plt.figure(figsize=(5.5, 5))
plt.contourf(x, y, rv.pdf(pos), levels=20, cmap="viridis")
plt.gca().set_aspect("equal")
plt.xlabel(r"$x_1$"); plt.ylabel(r"$x_2$")
plt.title(fr"2D Gaussian density, correlation $\rho$={rho}")
plt.colorbar(label="density")
plt.show()

```

